Bioinformatics, 2026, pp. 1–4

doi: DOI HERE
Advance Access Publication Date: Day Month Year
Paper

PAPER

RabbitBin: Binning Large-Scale Metagenomic Assemblies in Minutes on Multicore CPUs

Tong Zhang,[1] Zekun Yin,[1,*] Lifeng Yan,[2] Yijie Gao,[1] Bertil Schmidt[3] and Weiguo Liu[1]

[1]School of Software, Shandong University, Jinan, China, [2]Hetao Institute of Mathematics and Interdisciplinary Sciences (HIMIS), Shenzhen, China and [3]Institute for Computer Science, Johannes Gutenberg University, Mainz, Germany
[*]Corresponding author. zekun.yin@sdu.edu.cn
FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

Summary: We present RabbitBin, a fast metagenomic binning tool designed for large-scale metagenomic assemblies. RabbitBin constructs a sparse contig graph through dual-criterion screening based on coverage profiles and ProbMinHash similarity, and refines the resulting bins through K-means-based splitting and coverage-guided contig recruitment. RabbitBin accelerates BAM processing and contig comparison on multicore CPUs through parallel I/O, multithreading, and SIMD vectorization. On a server with 72 CPU cores, RabbitBin completes binning in 1.2 minutes for a 6.0-million-contig co-assembly of activated-sludge metagenomes with 308.1 GB of BAM files, achieving speedups of one to three orders of magnitude over MetaBAT2, QuickBin, SemiBin2, and COMEBin, while consuming significantly less memory at the same time. Across CAMI I and CAMI II benchmarks, RabbitBin achieves competitive binning quality.
Availability and Implementation: RabbitBin is available at `https://github.com/RabbitBio/RabbitBin` and archived at `https://doi.org/10.5281/zenodo.23261227`.
Contact: zekun.yin@sdu.edu.cn
Supplementary information: Supplementary data are available at *Bioinformatics* online.

Introduction

Metagenomic binning groups assembled contigs into genome bins, enabling the recovery of microbial genomes from complex communities. It supports microbial community analysis, pathogen identification, and the discovery of novel enzymes and metabolic pathways (Mallawaarachchi *et al.*, 2024). Existing metagenomic binning methods can be broadly divided into classical and deep-learning-based methods (Yazhini *et al.*, 2025). Classical methods cluster contigs based on sequence composition and coverage profiles. MetaBAT2 (Kang *et al.*, 2019) uses TNF and coverage profiles for graph-based clustering. QuickBin (Bushnell and Villada, 2026) uses GC content and coverage to select candidate pairs, and then compares their composition and k-mer similarities for merging. Deep-learning-based methods iteratively train neural networks to learn nonlinear representations from sequence composition and coverage profiles, rather than directly using similarities computed from these features for clustering. SemiBin2 uses self-supervised contrastive learning (Pan *et al.*, 2023), while COMEBin combines multi-view contrastive learning with Leiden clustering (Wang *et al.*, 2024). However, modern metagenomic assemblies can contain large numbers of contigs. At this scale, coverage computation from large BAM files and comparison among large numbers of contigs can require substantial compute and memory resources. Existing binning tools can therefore require long runtimes and high memory consumption on large metagenomic assemblies (Han *et al.*, 2025; Yazhini *et al.*, 2025). More efficient tools that maintain binning quality are therefore needed for such datasets.

To bridge this gap, we develop RabbitBin, which combines sparse graph construction that reduces pairwise computation with fine-grained optimization for multicore CPUs. RabbitBin constructs the graph using both coverage similarity and ProbMinHash-based sequence-composition similarity. It further uses coverage-based refinement to split heterogeneous bins and recruit unassigned contigs. Parallel BAM I/O, multithreading, and SIMD vectorization are applied to the computationally intensive stages of the workflow.

Methods

RabbitBin takes a contig assembly and BAM files as input. RabbitBin then constructs ProbMinHash (PMH) sketches for long contigs and computes coverage profiles for all contigs from the BAM files (Fig. 1(a)). RabbitBin uses long contigs for initial graph construction to reduce graph size and avoid unreliable composition similarity from short contigs, while short contigs are reserved for subsequent recruitment. For initial binning, RabbitBin screens pairs of long contigs through coverage filtering and PMH neighbour selection, computes edge weights from coverage profiles, and applies Fisher-style label propagation to generate the initial bins (Fig. 1(b)). RabbitBin

© The Author 2026. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

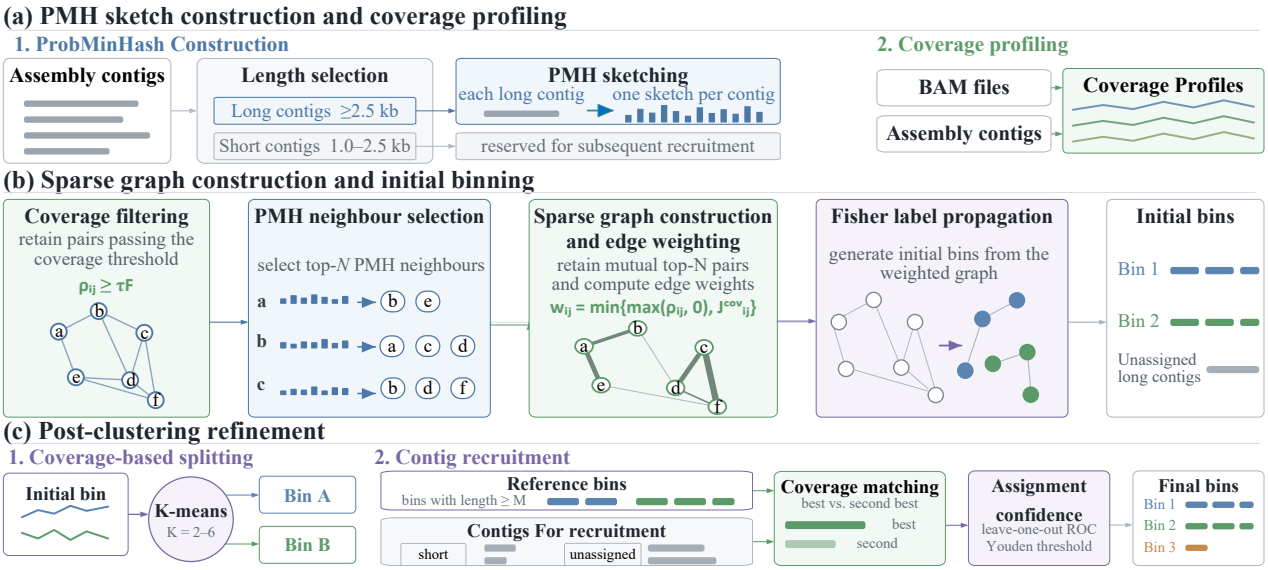


Fig. 1. Overview of RabbitBin. (a) RabbitBin constructs PMH sketches for long contigs and computes coverage profiles from BAM files. (b) RabbitBin screens contig pairs using coverage filtering and PMH neighbour selection, computes edge weights from coverage profiles, and applies Fisher-style label propagation to generate the initial bins. (c) RabbitBin refines the initial bins by coverage-based splitting and recruits unassigned long contigs and short contigs using assignment confidence from the best and second-best coverage matches.

then refines the initial bins by first splitting bins using K-means clustering on their coverage profiles and then recruiting short contigs and unassigned long contigs using assignment confidence with an ROC-derived Youden-based recruitment threshold (Fig. 1(c)). Computationally intensive stages are further optimized for multicore CPUs.

Dual-criterion contig-pair screening and coverage-based edge weighting

To reduce graph size for subsequent clustering, RabbitBin screens long-contig pairs using coverage similarity followed by PMH-based sequence-composition similarity (see Supplementary Section 1). RabbitBin computes coverage profiles for all contigs from the input BAM files, with long-contig profiles used for graph construction and short-contig profiles used later for recruitment. For S BAM samples, RabbitBin computes regular and MAPQ-filtered coverage for each sample by default, resulting in $D = 2S$ coverage features (see Supplementary Section 2). For two long contigs i and j, RabbitBin computes the Spearman rank correlation $\rho_{ij} = \mathrm{corr}(\mathrm{rank}(\mathbf{d}_i), \mathrm{rank}(\mathbf{d}_j))$ between their coverage profiles $\mathbf{d}_i = (d_{i1}, \ldots, d_{iD})$ and $\mathbf{d}_j = (d_{j1}, \ldots, d_{jD})$, where D denotes the number of coverage features. RabbitBin uses a Fisher-derived threshold of $\tau_F = 0.71533$ for coverage screening (Zaykin *et al.*, 2007). This threshold removes pairs with weak coverage support that could reduce the combined label score during subsequent label propagation (see Supplementary Section 2.2). Contig pairs with $\rho_{ij} < \tau_F$ are removed. For the remaining long contig pairs, RabbitBin further compares sequence composition using ProbMinHash (PMH) (Ertl, 2020). RabbitBin applies PMH to contig sequences by treating canonical 4-mers as set elements and computing a PMH weight for each 4-mer. For a long contig c, the PMH weight of each canonical 4-mer x is computed as

$$w_c(x) = \frac{f_c(x)/\sum_y f_c(y)}{\prod_{\ell=1}^{4} \pi_c(x_\ell) + \mathbf{1}_{x \neq rc(x)} \prod_{\ell=1}^{4} \pi_c(rc(x)_\ell)},$$

where $f_c(x)$ is the occurrence count of x, and $\pi_c(b)$ is the frequency of nucleotide b in contig c. For each long contig, RabbitBin constructs a PMH sketch from its weighted canonical 4-mers, with a default sketch size of 500. For each contig pair retained after coverage screening, RabbitBin uses the PMH sketches to estimate their probability Jaccard similarity. For each long contig, RabbitBin selects the top-N most similar contigs according to PMH similarity and retains a pair only if each contig is among the other's top-N contigs. RabbitBin uses $N = 200$ by default, which empirically provides stable bin recovery while keeping the graph sparse (see Supplementary Section 1.4).

For each contig pair that passes both coverage and PMH screening, RabbitBin defines the edge weight as $w_{ij} = \min\{\max(\rho_{ij}, 0), J_{ij}^{\mathrm{cov}}\}$, where J_{ij}^{cov} is the weighted Jaccard similarity between the normalized coverage profiles. Edges with $w_{ij} \geq \tau_F$ are retained.

When only one or two samples are available, Spearman correlation is not suitable because the coverage profiles provide limited rank information. RabbitBin computes the edge weight as the mean min-to-max ratio across the D coverage features. The detailed formulation is provided in Supplementary Section 2.1.

Fisher-style label propagation

RabbitBin clusters the weighted sparse graph using Fisher-style label propagation. Each contig is initially assigned a unique label. RabbitBin randomly orders the contigs once, uses the same order in all propagation rounds, and immediately applies each updated label when processing subsequent contigs in the same round. For a contig v, let $\mathcal{N}_k(v)$ denote its neighbours with label k. RabbitBin computes the joint support for label k by combining the corresponding edge weights using the Fisher-style aggregation score

$$T_{v,k} = -2 \sum_{u \in \mathcal{N}_k(v)} \log(1 - w_{vu}).$$

Because different labels may have different numbers of supporting neighbours, RabbitBin converts $T_{v,k}$ to a comparable label score as

$$S_{v,k} = F_{\chi^2_{2m_{v,k}}}(T_{v,k}), \qquad m_{v,k} = |\mathcal{N}_k(v)|,$$

where $F_{\chi^2_{2m_{v,k}}}$ is the cumulative distribution function of the chi-square distribution with $2m_{v,k}$ degrees of freedom. RabbitBin assigns the label with the highest score to v. Label propagation continues until no label changes during a complete round. Contigs with the same final label form an initial bin.

Post-clustering refinement

RabbitBin refines the initial bins to separate bins with heterogeneous coverage profiles and recruit short contigs and unassigned long contigs. RabbitBin first applies K-means to bins with a total length of at least M (200 kb by default). For each eligible bin, RabbitBin evaluates $K = 2, \ldots, 6$ and selects the clustering with the highest mean silhouette score. The selected partition is accepted only when its silhouette score is at least 0.70; otherwise, the bin remains unchanged (see Supplementary Section 3.1).

After splitting, bins with a total length of at least M are used as reference bins for contig recruitment. For each short contig or unassigned long contig i, RabbitBin compares its coverage profile with each reference bin B and computes a similarity $q(i, B)$. For datasets with at least three samples, $q(i, B)$ is the cosine similarity between the rank-transformed coverage profile of i and the coverage representation of B. Let $q_i^{(1)}$ and $q_i^{(2)}$ denote the best and second-best similarities, respectively. RabbitBin computes the assignment confidence as

$$C_i = \log\left(\frac{1 - q_i^{(2)}}{1 - q_i^{(1)}}\right),$$

where a larger C_i indicates a stronger preference for the best-matching bin. RabbitBin determines the recruitment threshold through leave-one-out matching on contigs in the reference bins. For each contig, RabbitBin removes it from its original bin and recomputes its similarities to all reference bins. The resulting C_i is labeled positive when the original bin remains the best match and negative otherwise. These labeled scores define an ROC curve, from which RabbitBin determines a Youden-based recruitment threshold. Contigs with C_i at or above this threshold are assigned to their best-matching bins. Details of coverage similarity and leave-one-out calibration are provided in Supplementary Section 3.2, with the single- and two-sample formulation described in Supplementary Section 2.1.

Efficient multicore implementation

RabbitBin applies fine-grained performance optimizations to input processing and graph construction, which account for most of the runtime. During input processing, RabbitBin uses our previously developed RabbitBAM (Yan *et al.*, 2025) to read BAM files in parallel and compute per-contig coverage. RabbitBin combines FASTA parsing and PMH sketch construction in a single pass and parallelizes sketch construction across contigs on multicore CPUs. For graph construction, RabbitBin exploits parallelism at two levels. Contig-pair comparisons are distributed across OpenMP threads, while each comparison is further accelerated by SIMD vectorization over PMH sketch registers and, for multi-sample datasets, over coverage profiles during correlation computation (Supplementary Section 4).

Results

We benchmark RabbitBin against MetaBAT2 v2.18, QuickBin (BBTools v40.02), SemiBin2 v2.3.0, and COMEBin v1.0.4. The MiDAS-AS activated-sludge co-assembly is used to evaluate large-scale computational performance on real metagenomic data. It contains 6.0 million contigs and 308.1 GB of coordinate-sorted BAM files (Han *et al.*, 2025). Because MiDAS-AS has no gold-standard genome assignments, bin quality is evaluated using CheckM2 (Chklovski *et al.*, 2023). Bins are classified as HQ, MQ, and LQ using completeness/contamination thresholds of 90/5%, 70/10%, and 50/10%, respectively. The marine, plant-associated, and strain-madness datasets from CAMI II are used to evaluate binning quality with gold-standard contig-to-genome assignments (Meyer *et al.*, 2022). Recovered bins are evaluated using AMBER (Meyer *et al.*, 2018) with the corresponding completeness/purity criteria. To further evaluate RabbitBin in single- and two-sample settings, we use the CAMI I Low and Medium datasets, respectively, with the corresponding results reported in Supplementary Section 5.4. Bins shorter than 200 kb are excluded from quality evaluation. All experiments are performed on a server equipped with 72 CPU cores and an NVIDIA RTX A6000 GPU. GPU acceleration is enabled for tools that support GPU computation, while RabbitBin runs entirely on CPUs. Detailed dataset preparation and hardware configuration are provided in Supplementary Section 5.

Table 1 summarizes the benchmark results. On the three CAMI II datasets, RabbitBin recovers 286, 84, and 33 HQ bins on the marine, plant-associated, and strain-madness datasets, respectively. Among the evaluated classical methods, RabbitBin recovers the largest number of HQ bins on all three datasets, but fewer than the deep-learning-based methods. Further analysis shows that directly using sequence-composition and coverage similarities, as commonly done by classical methods, may miss relationships among contigs from the same genome (Supplementary Section 5.3). Deep-learning-based methods can achieve higher binning quality by learning contig representations, but their higher computational cost can limit their practical use on large-scale metagenomic assemblies. In contrast, RabbitBin completes binning on the three CAMI II datasets in 11–36 s, achieving two to three orders of magnitude speedups over these methods. On the real-world MiDAS-AS activated-sludge co-assembly, RabbitBin completes binning in 73 s, achieving 13.4×–728.4× speedups over MetaBAT2, QuickBin, and SemiBin2 while recovering the largest number of HQ bins (265 versus 234, 169, and 237, respectively). COMEBin does not complete within 2 days. In addition, RabbitBin uses only 9.3 GB of memory, reducing memory consumption by at least 77% compared with the other tools. The dual-criterion contig-pair screening keeps the contig graph sparse and avoids storing a dense graph. RabbitBin also scales nearly linearly up to 18 threads and achieves a 46.9× speedup with 72 threads relative to single-threaded execution (Supplementary Section 5.5). In single- and two-sample settings, RabbitBin achieves 5.3×–765.3× speedups over the evaluated tools on the CAMI I Low and Medium datasets, respectively, while maintaining comparable binning quality (Supplementary Section 5.4). Overall, RabbitBin maintains competitive binning quality while substantially reducing execution time and memory consumption, making it practical for large-scale metagenomic binning.

Table 1. Performance evaluation of RabbitBin on the MiDAS-AS co-assembly and CAMI II datasets.

Dataset	Tool	HQ	MQ	LQ	Memory (GB)	Runtime (s)	Speedup
CAMI II marine	MetaBAT2	139	164	176	3.3	70	6.4×
	QuickBin	237	285	322	136.4	174	15.8×
	SemiBin2	276	379	415	27.8	5402	491.1×
	COMEBin	**313**	**402**	**456**	15.2	20228	1838.9×
	RabbitBin	286	353	394	**1.3**	**11**	1.0×
CAMI II plant-associated	MetaBAT2	64	70	72	10.6	75	4.2×
	QuickBin	74	86	96	140.7	262	14.6×
	SemiBin2	**91**	**102**	**109**	29.5	3661	203.4×
	COMEBin	85	96	101	13.7	15667	870.4×
	RabbitBin	84	94	103	**1.1**	**18**	1.0×
CAMI II strain-madness	MetaBAT2	12	14	18	2.0	59	1.6×
	QuickBin	19	26	26	134.2	364	10.1×
	SemiBin2	**49**	65	66	38.7	4389	121.9×
	COMEBin	48	**69**	**81**	6.4	5460	151.7×
	RabbitBin	33	38	44	**0.7**	**36**	1.0×
MiDAS-AS	MetaBAT2	234	926	1160	41.1	5258	72.0×
	QuickBin	169	662	989	184.1	981	13.4×
	SemiBin2	237	**1217**	**1454**	101.5	53172	728.4×
	COMEBin	–	–	–	–	>172800[a]	>2367.1×
	RabbitBin	**265**	913	1257	**9.3**	**73**	1.0×

[a] COMEBin did not complete within 2 days on MiDAS-AS; therefore, its runtime and speedup are reported as lower bounds.

Conclusion

RabbitBin addresses the computational challenges associated with binning million-contig metagenomic assemblies by combining sparse graph construction with fine-grained performance optimizations while maintaining competitive binning quality. By enabling efficient large-scale and multi-sample metagenomic binning on multicore CPUs, RabbitBin facilitates genome recovery from increasingly large metagenomic datasets and subsequent genome-resolved analyses.

Acknowledgements

OpenAI ChatGPT (GPT-5.6 Sol) was used for language refinement of the manuscript.

References

Bushnell, B. and Villada, J. C. (2026). Deployable high-fidelity metagenome binning at scale with quickbin. *Communications Biology*, **9**(1), 1135.

Chklovski, A. *et al.* (2023). Checkm2: a rapid, scalable and accurate tool for assessing microbial genome quality using machine learning. *Nature methods*, **20**(8), 1203–1212.

Ertl, O. (2020). Probminhash–a class of locality-sensitive hash algorithms for the (probability) jaccard similarity. *IEEE Transactions on Knowledge and Data Engineering*, **34**(7), 3491–3506.

Han, H. *et al.* (2025). Benchmarking metagenomic binning tools on real datasets across sequencing platforms and binning modes. *Nature Communications*, **16**(1), 2865.

Kang, D. D. *et al.* (2019). Metabat 2: an adaptive binning algorithm for robust and efficient genome reconstruction from metagenome assemblies. *PeerJ*, **7**, e7359.

Mallawaarachchi, V. *et al.* (2024). Solving genomic puzzles: computational methods for metagenomic binning. *Briefings in Bioinformatics*, **25**(5), bbae372.

Meyer, F. *et al.* (2018). Amber: assessment of metagenome binners. *Gigascience*, **7**(6), giy069.

Meyer, F. *et al.* (2022). Critical assessment of metagenome interpretation: the second round of challenges. *Nature methods*, **19**(4), 429–440.

Pan, S. *et al.* (2023). Semibin2: self-supervised contrastive learning leads to better mags for short-and long-read sequencing. *Bioinformatics*, **39**(Supplement_1), i21–i29.

Wang, Z. *et al.* (2024). Effective binning of metagenomic contigs using contrastive multi-view representation learning. *Nature Communications*, **15**(1), 585.

Yan, L. *et al.* (2025). Rabbitbam: Accelerating bam file manipulation on multi-core platforms. *IEEE Transactions on Computational Biology and Bioinformatics*, **22**(5), 2320–2326.

Yazhini, A. *et al.* (2025). Evaluation of metagenome binning: advances and challenges. *Briefings in Bioinformatics*, **26**(6), bbaf617.

Zaykin, D. V. *et al.* (2007). Combining p-values in large-scale genomics experiments. *Pharmaceutical Statistics: The Journal of Applied Statistics in the Pharmaceutical Industry*, **6**(3), 217–226.